

IT3280 – THỰC HÀNH KIẾN TRÚC MÁY TÍNH

BÁO CÁO THỰC HÀNH

BÀI 12: Bộ nhớ đệm nhanh – Cache memory

Họ và tên: Nguyễn Hồng Khôi

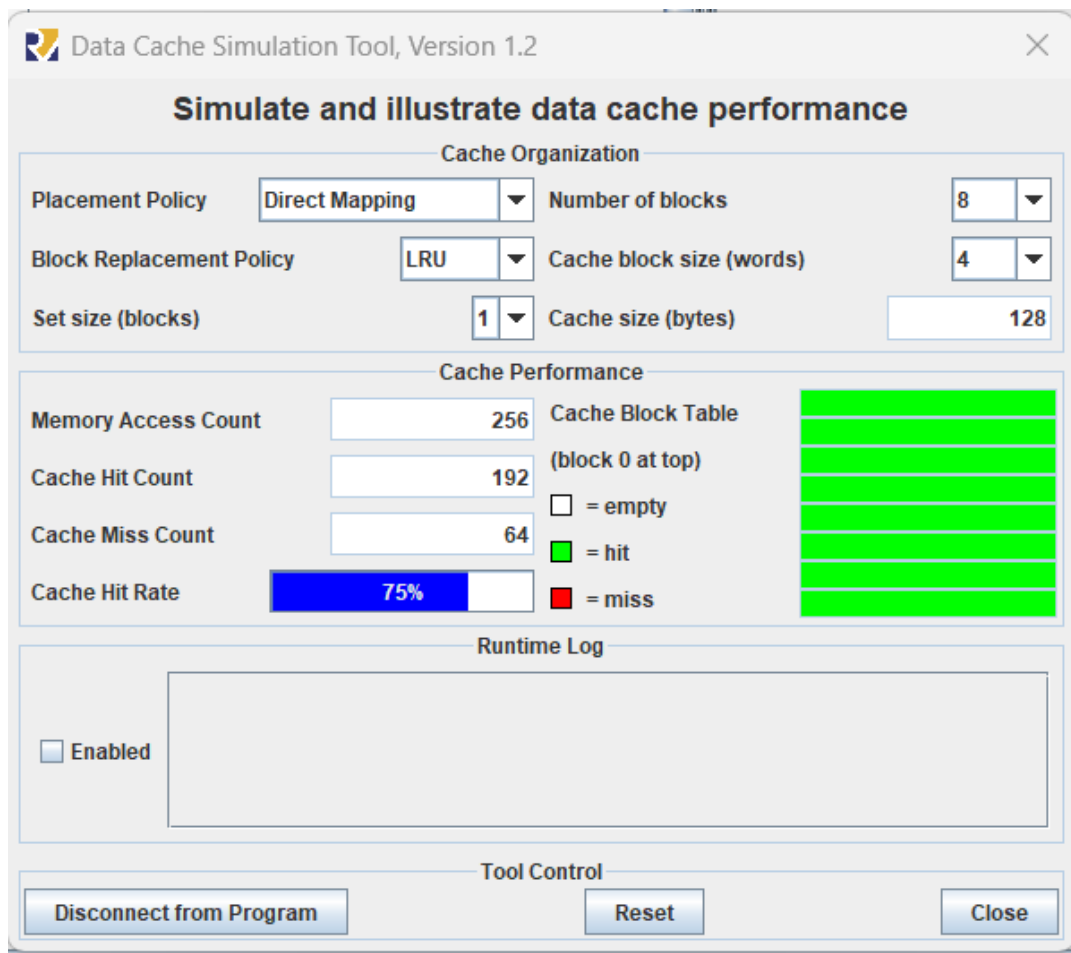
MSSV: 202416956

Assignment 1:

Câu 7. Tỷ lệ Cache Hit

Hiệu suất đạt ngưỡng 75% chứng minh kiến trúc bộ nhớ đệm đang tối ưu hóa rất tốt tiến trình vận hành của hệ thống. Thành quả này bắt nguồn từ việc chương trình quét dữ liệu tuần tự theo đúng sơ đồ sắp đặt trong bộ nhớ vật lý. Nhờ vậy, ngay khi một ô nhớ được kích hoạt, các phần tử kế thừa nó phần lớn đã được gom sẵn vào khối cache vừa khởi tạo.

Cụ thể, ở thiết lập mặc định, mỗi block định hình dung lượng 4 words, cho phép gom đồng bộ 4 phần tử liền kề từ RAM sang cache trong một chu kỳ truy xuất. Thao tác đọc đầu tiên hướng vào một khối mới luôn dẫn đến hiện tượng compulsory miss (lỗi nạp bắt buộc) do vùng nhớ này hoàn toàn trống rỗng. Ngay sau pha trượt này, cơ chế Direct Mapping (ánh xạ trực tiếp) sẽ chịu trách nhiệm định vị và đẩy toàn bộ khối dữ liệu đó vào bộ nhớ đệm. Chu trình này thiết lập một quy luật lặp lại đều đặn: cứ 1 lần miss để khởi tạo block sẽ mở đường cho 3 lần hit liên tiếp phía sau nhờ tận dụng triệt để các ô nhớ lân cận. Áp dụng vào quy mô 256 pha truy xuất, hệ thống ghi nhận chính xác 192 lần hit đi kèm 64 lần miss.



-> Kết quả thu được phù hợp với mô hình lý thuyết của truy cập tuần tự theo hàng.

Câu 8: Dự đoán tỷ lệ cache hit nếu kích thước khối thay đổi

Trong kiến trúc máy tính, việc điều chỉnh kích thước block sẽ định hình lại lượng dữ liệu được nạp vào cache trong mỗi chu kỳ cấu hình. Khi mở rộng dung lượng block, hệ thống tận dụng tối đa tính cục bộ không gian bằng cách gom toàn bộ các phần tử lân cận vào bộ nhớ đệm trong một lần nạp. Trái lại, block quá nhỏ sẽ ép CPU phải tăng tần suất giao tiếp với bộ nhớ chính, gây thắt nút cổ chai. Đối với các thuật toán có hành vi truy xuất tuyến tính (đọc tuần tự các ô nhớ kế tiếp), kích thước block chính là biến số quyết định hiệu năng vận hành của cache.

Minh chứng rõ nhất là khi ta nâng kích cỡ block từ 4 lên 8 words. Sự thay đổi này chắc chắn sẽ đẩy tỷ lệ Cache Hit gia tăng mạnh mẽ, bởi số lượng dữ liệu đi kèm sau mỗi pha xử lý trượt (miss) đã nhiều hơn trước. Về mặt kỹ thuật, cấu hình 8 words cho phép một lần miss duy nhất kéo theo cả khối 8 phần tử liên tiếp vào cache. Khi chương trình quét dữ liệu theo hàng, ngoại trừ phần tử đầu tiên gây miss, 7 phần tử kế sau đã nằm sẵn trong bộ nhớ đệm và lập tức biến thành hit. Mô hình này thiết lập một chu kỳ lý tưởng gồm 1 miss và 7 hit, giúp tối ưu hóa hiệu suất hit rate đạt ngưỡng lý thuyết là 87.5%.

Data Cache Simulation Tool, Version 1.2

Simulate and illustrate data cache performance

Cache Organization

Placement Policy: Direct Mapping (dropdown) Number of blocks: 8 (dropdown)

Block Replacement Policy: LRU (dropdown) Cache block size (words): 8 (dropdown)

Set size (blocks): 1 (dropdown) Cache size (bytes): 256 (text box)

Cache Performance

Memory Access Count: 256 (text box) Cache Block Table: (table with 8 green blocks, 0 empty blocks)

Cache Hit Count: 224 (text box) (block 0 at top)

Cache Miss Count: 32 (text box) ☐ = empty

Cache Hit Rate: 88% (progress bar) ☒ = hit

Runtime Log

☐ Enabled

Tool Control

Disconnect from Program Reset Close

Xu hướng này cũng giải thích cho kịch bản thu hẹp kích thước block từ 4 xuống còn 2 words. Hệ quả tất yếu là tỷ lệ Cache Hit sẽ lao dốc, do dung lượng dữ liệu dự phòng được kéo về sau mỗi pha xử lý trượt (miss) đã bị cắt giảm đáng kể. Xét về bản chất vận hành, cấu hình 2 words giới hạn mỗi lần miss chỉ có thể nạp thêm duy nhất một phần tử lân cận vào bộ nhớ đệm. Lúc này, chu kỳ truy xuất của chương trình bị ép vào mô hình tối giản: cứ 1 lần miss sẽ đi kèm cấu trúc 1 lần hit. Kết quả là hiệu suất hit rate lý

thuyết bị kéo tụt xuống mức 50%, minh chứng rõ ràng cho việc block size quá hẹp sẽ bóp nghẹt khả năng khai thác tính cục bộ không gian của hệ thống.

Data Cache Simulation Tool, Version 1.2

Simulate and illustrate data cache performance

Cache Organization

Placement Policy: **Direct Mapping** | Number of blocks: **8**

Block Replacement Policy: **LRU** | Cache block size (words): **2**

Set size (blocks): **1** | Cache size (bytes): **64**

Cache Performance

Memory Access Count: **256** | Cache Block Table (block 0 at top):

Cache Hit Count: **128** | ☐ = empty

Cache Miss Count: **128** | ☒ = hit

Cache Hit Rate: **50%** | ☐ = miss

Runtime Log

☐ Enabled

Tool Control

Disconnect from Program | **Reset** | **Close**

-> Kết quả thực nghiệm phù hợp với phân tích lý thuyết.

Câu 9: Kiểm nghiệm kết quả

Số liệu thực nghiệm thu được ghi nhận sự đồng nhất gần như tuyệt đối với các kết quả tính toán trên lý thuyết. Điều này khẳng định thuật toán vận hành của trình giả lập đã tái hiện rất trực quan và chính xác cơ chế xử lý của bộ nhớ đệm trong môi trường phần cứng thực tế. Những chênh lệch không đáng kể giữa số liệu dự báo và kết quả đo đạc hoàn toàn không làm thay đổi bản chất của các nguyên lý kiến trúc máy tính.

Chẳng hạn, khi cấu hình thông số Block size ở mức 8 words, hệ thống mô phỏng báo cáo chỉ số Cache Hit Rate dao động trong khoảng từ 87% đến 88%, bám rất sát ngưỡng lý thuyết là 87.5%. Sự sai lệch cực nhỏ này chỉ đơn thuần xuất phát từ cơ chế làm tròn số thập phân trong giao diện hiển thị của công cụ.


Data Cache Simulation Tool, Version 1.2
✕

Simulate and illustrate data cache performance

Cache Organization

Placement Policy	Direct Mapping	Number of blocks	8
Block Replacement Policy	LRU	Cache block size (words)	8
Set size (blocks)	1	Cache size (bytes)	256

Cache Performance

Memory Access Count	256	Cache Block Table (block 0 at top)
Cache Hit Count	224	
Cache Miss Count	32	
Cache Hit Rate	88%	

☐ = empty
☒ = hit
☐ = miss

Runtime Log

☐ Enabled

Tool Control

Disconnect from Program
Reset
Close

- Với Block size = 2 words: Cache Hit Rate đạt 50%, đúng với chu kỳ 1 miss - 1 hit của khối 2 words.

Data Cache Simulation Tool, Version 1.2

Simulate and illustrate data cache performance

Cache Organization

Placement Policy: Direct Mapping Number of blocks: 8

Block Replacement Policy: LRU Cache block size (words): 2

Set size (blocks): 1 Cache size (bytes): 64

Cache Performance

Memory Access Count: 256 Cache Block Table (block 0 at top)

Cache Hit Count: 128 ☐ = empty

Cache Miss Count: 128 ☒ = hit

Cache Hit Rate: 50% ☐ = miss

Runtime Log

☐ Enabled

Tool Control

Disconnect from Program Reset Close

Câu 11: Hiệu năng truy cập bộ nhớ cache đối với chương trình này là bao nhiêu?

- **Hệ quả vận hành:** Chỉ số Cache Hit Rate chạm mức 0%, đồng nghĩa với việc tỷ lệ Cache Miss bị đẩy lên tuyệt đối 100%.
- **Bản chất kỹ thuật:** Hiện tượng này xảy ra do chương trình thiết lập một lộ trình truy xuất có bước nhảy địa chỉ (stride) quá lớn giữa các chu kỳ đọc liên tiếp. Dù cơ chế phần cứng của bộ nhớ đệm vẫn kích hoạt lệnh nạp dữ liệu theo từng khối, song phần lớn các ô nhớ đi kèm trong khối đó lại bị ghi đè hoặc loại bỏ trước khi kịp sử dụng. Hệ quả là vai trò tối ưu hóa tốc độ của cache hoàn toàn bị triệt tiêu. Trạng thái Hit Rate bằng 0% này chính là kịch bản tồi tệ nhất trong kiến trúc máy tính, xuất phát từ sự bất đối xứng nghiêm trọng giữa sơ đồ định tuyến của thuật toán và cấu trúc sắp đặt dữ liệu vật lý.

Cụ thể, với thiết lập block size bằng 4 words, pha truy cập vào ô nhớ `data[0][0]` sẽ kích hoạt một lỗi trượt (miss), buộc cache phải kéo theo cụm dữ liệu từ `data[0][0]` đến `data[0][3]` về bộ nhớ đệm. Khối dữ liệu này sẽ chỉ phát huy giá trị nếu chương trình duy trì thao tác đọc trên cùng một hàng. Trớ trêu thay, lệnh điều hướng ngay sau đó lại lập tức nhảy sang ô nhớ `data[1][0]` – một vị trí nằm hoàn toàn ngoài phạm vi của block vừa nạp. Do bước nhảy địa chỉ vượt quá giới hạn lưu trữ của một khối đơn lẻ, phân vùng dữ liệu dự phòng trong cache trở nên vô dụng. Tiến trình này lặp lại liên tục ở mọi chu kỳ quét, biến mỗi phần tử được gọi thành một pha miss độc lập. Có thể khẳng định, phương thức duyệt mảng theo cột đã phá hủy hoàn toàn tính cục bộ không gian (Spatial Locality) đối với quy cách định dạng row-

major (lưu trữ theo hàng ưu tiên) của mảng hai chiều, khiến bộ nhớ đệm mất đi khả năng giảm thiểu độ trễ cho toàn hệ thống.

Data Cache Simulation Tool, Version 1.2

Simulate and illustrate data cache performance

Cache Organization

Placement Policy: **Direct Mapping** | Number of blocks: **8**

Block Replacement Policy: **LRU** | Cache block size (words): **4**

Set size (blocks): **1** | Cache size (bytes): **128**

Cache Performance

Memory Access Count: **256** | Cache Block Table (block 0 at top)

Cache Hit Count: **0** | ☐ = empty

Cache Miss Count: **256** | ☒ = hit

Cache Hit Rate: **0%** | ☒ = miss

Runtime Log

☐ Enabled

Tool Control

Disconnect from Program | **Reset** | **Close**

-> Kết quả chính xác và phản ánh rõ ảnh hưởng của thứ tự duyệt mảng đến hiệu năng cache.

Câu 12: Kết quả chạy

Data Cache Simulation Tool, Version 1.2

Simulate and illustrate data cache performance

Cache Organization

Placement Policy: Direct Mapping Number of blocks: 8

Block Replacement Policy: LRU Cache block size (words): 16

Set size (blocks): 1 Cache size (bytes): 512

Cache Performance

Memory Access Count: 256 Cache Block Table (block 0 at top)

Cache Hit Count: 0 ☐ = empty

Cache Miss Count: 256 ☒ = hit

Cache Hit Rate: 0% ☒ = miss

Runtime Log

☐ Enabled

Tool Control

Disconnect from Program Reset Close

Câu 14: Chạy lại chương trình khi thay đổi cấu hình

Đánh giá hiệu năng truy cập trên cửa sổ cũ với cấu hình Block size = 16 words và Number of blocks = 8:

- **Hệ quả thực nghiệm:** Chỉ số Cache Hit Rate vẫn dậm chân ở mức 0%.
- **Phân tích nguyên nhân:** Bản chất của kết quả này chứng minh một nguyên lý cốt lõi: mở rộng kích thước block không phải là chiếc chìa khóa vạn năng để nâng cao hiệu suất hệ thống. Trong kịch bản này, mặc dù dung lượng mỗi block đã được nâng lên đủ lớn để bao trọn toàn bộ một hàng dữ liệu, nhưng hệ thống lại gặp phải điểm nghẽn ở số lượng block (Number of blocks) quá ít. Sự giới hạn này buộc nhiều hàng dữ liệu khác nhau trong bộ nhớ phải tranh chấp và chia sẻ chung một vị trí ánh xạ duy nhất trên cache. Hệ quả là các khối dữ liệu liên tục rơi vào vòng lặp bị ghi đè và trục xuất. Trước khi bộ vi xử lý kịp quay vòng để tái sử dụng vùng dữ liệu cũ, cấu trúc đó đã bị thay thế bởi một dòng lệnh mới, gây ra hiện tượng xung đột ánh xạ (Conflict Miss) nghiêm trọng và triệt tiêu hoàn toàn vai trò tăng tốc của bộ nhớ đệm.

Data Cache Simulation Tool, Version 1.2

Simulate and illustrate data cache performance

Cache Organization

Placement Policy: **Direct Mapping** Number of blocks: **8**

Block Replacement Policy: **LRU** Cache block size (words): **16**

Set size (blocks): **1** Cache size (bytes): **512**

Cache Performance

Memory Access Count: **256** Cache Block Table (block 0 at top)

Cache Hit Count: **0** ☐ = empty

Cache Miss Count: **256** ☒ = hit

Cache Hit Rate: **0%** ☒ = miss

Runtime Log

☐ Enabled

Tool Control

Disconnect from Program **Reset** **Close**

- **Nhận định sơ bộ:** Hiện tượng trên phản ánh chính xác bản chất của lỗi xung đột ánh xạ (Conflict Miss) đặc trưng trong cấu trúc Direct-Mapped Cache.

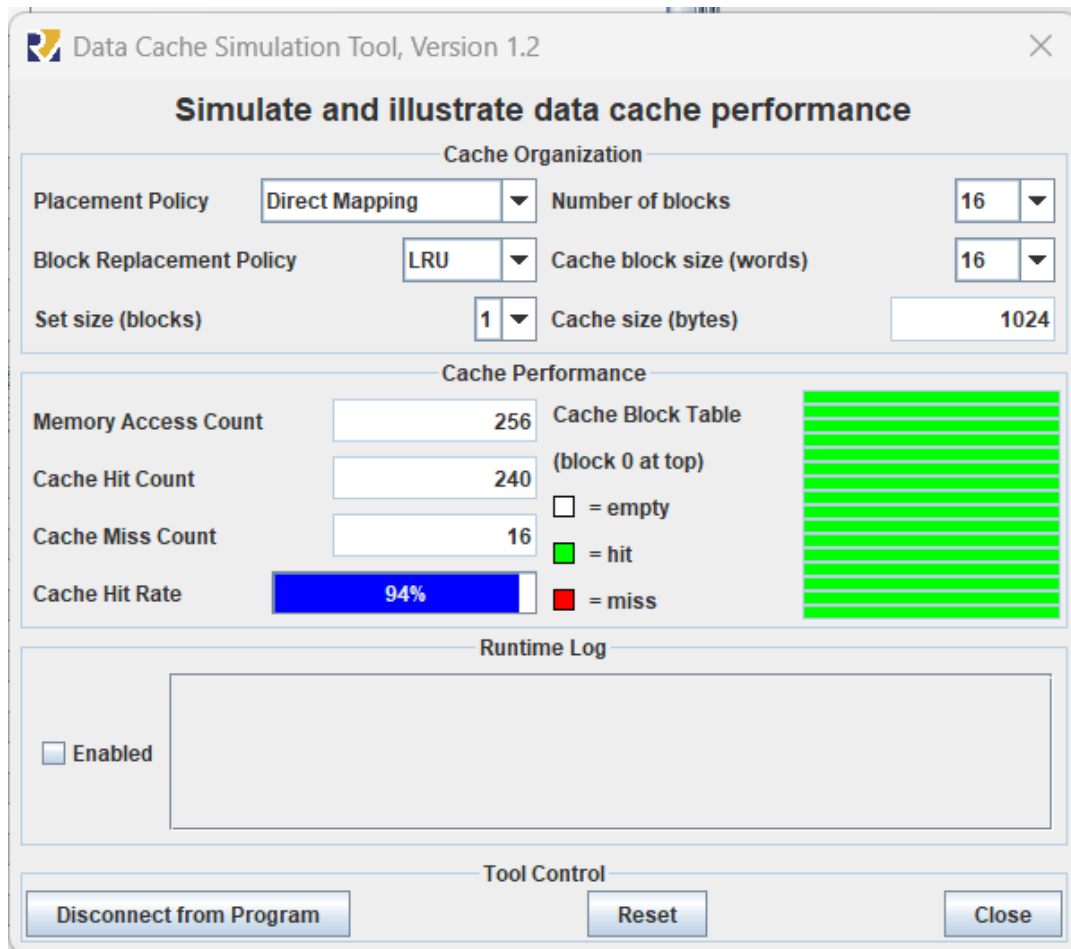
Đánh giá hiệu năng truy cập trên cửa sổ mới với cấu hình Block size = 16 words và Number of blocks = 16:

- **Hệ quả thực nghiệm:** Chỉ số Cache Hit Rate tăng vọt lên ngưỡng lý tưởng 93.75%.
- **Phân tích nguyên nhân:** Khi không gian lưu trữ được mở rộng vừa vặn để dung nạp toàn bộ cấu trúc ma trận, bộ nhớ đệm không còn rơi vào tình trạng phải liên tục trục xuất các khối dữ liệu vừa nạp. Trải qua chu kỳ khởi tạo sơ cấp, hầu như toàn bộ tài nguyên cốt lõi đều đã định cư an toàn trong cache. Các thao tác truy xuất ở chu kỳ sau chỉ đơn thuần là đọc lại vùng dữ liệu sẵn có, tạo bước đệm cho tỷ lệ hit tăng trưởng mạnh mẽ. Kịch bản này là một minh chứng trực quan cho thấy việc tối ưu hóa dung lượng tổng thể của cache có thể giải phóng đáng kể năng lực vận hành của hệ thống.

Xét về tiến trình chi tiết, ở lượt quét cột đầu tiên, thao tác đọc phần tử mở đầu của mỗi hàng sẽ kích hoạt một pha miss bắt buộc do hàng đó chưa từng được nạp vào cache. Ngay sau sự cố trượt này, toàn bộ phân vùng hàng gồm 16 phần tử sẽ được gom gọn vào một block độc lập. Kết thúc chu kỳ duyệt cột đầu tiên, toàn bộ 16 hàng dữ liệu của ma trận đã được ánh xạ thành công và định vị sẵn trong bộ nhớ đệm.

Bước sang các cột kế tiếp, dù chương trình vẫn duy trì lộ trình quét theo chiều dọc, nhưng tất cả các ô nhớ được gọi đến lúc này đều nằm trong các hàng đã được đồng bộ từ trước. Hệ quả là hầu như toàn bộ

tiến trình truy xuất đều được đáp ứng tức thì dưới dạng cache hit. Với tổng quy mô 256 lần truy cập, hệ thống chỉ ghi nhận vốn vẹn 16 pha miss (tương ứng với 16 chu kỳ nạp sơ cấp cho 16 hàng vật lý), trong khi số pha hit được đẩy lên mức 240 ($256 - 16$).



-> Tỷ lệ $240/256 = 93.75\%$, trùng với kết quả mô phỏng sau khi làm tròn.

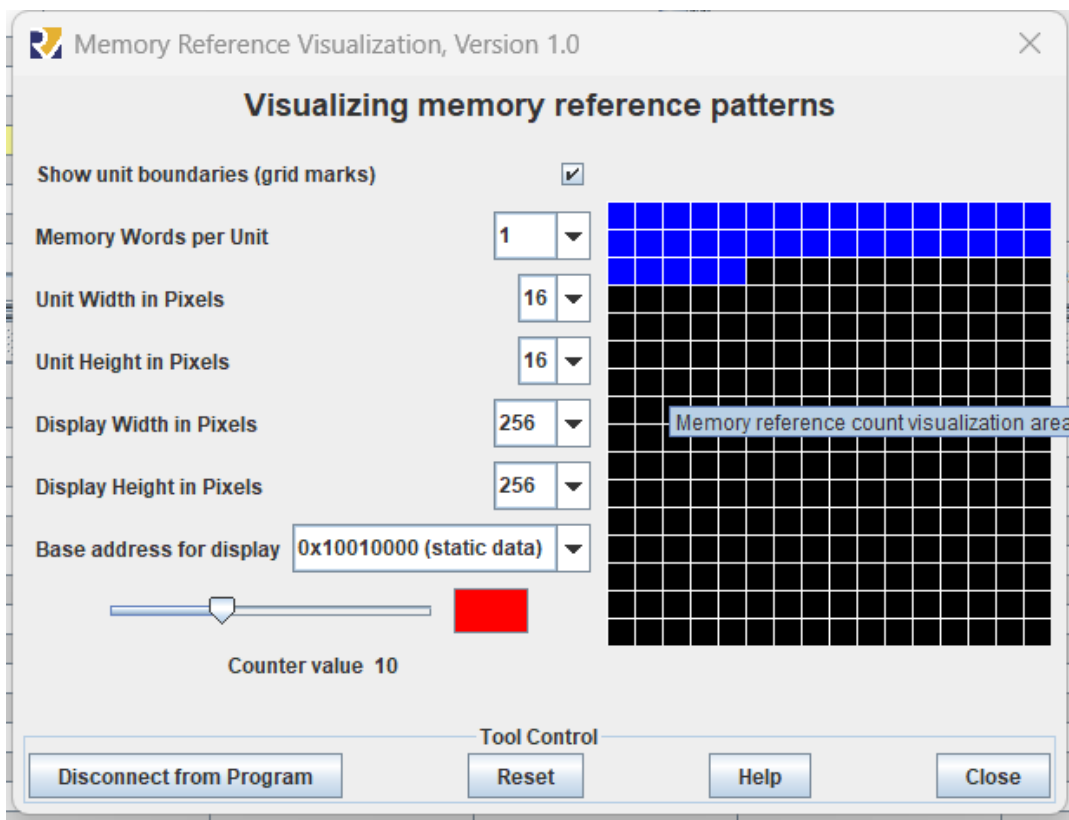
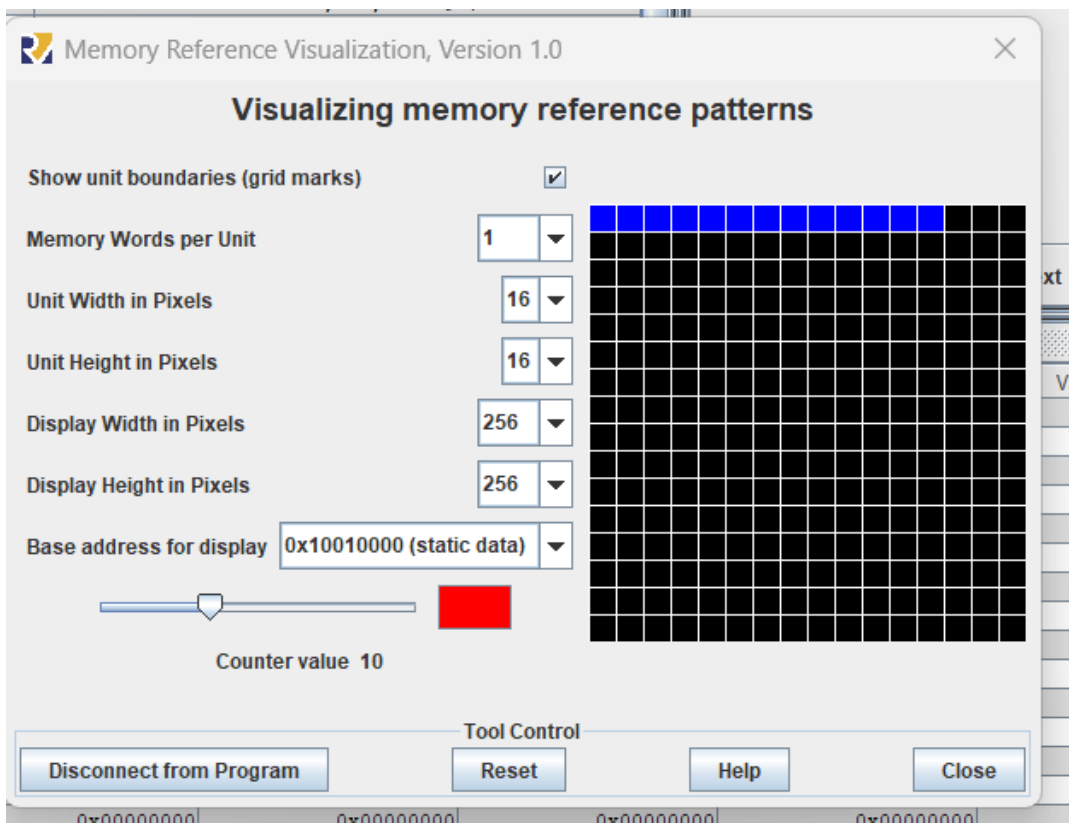
Assignment 2:

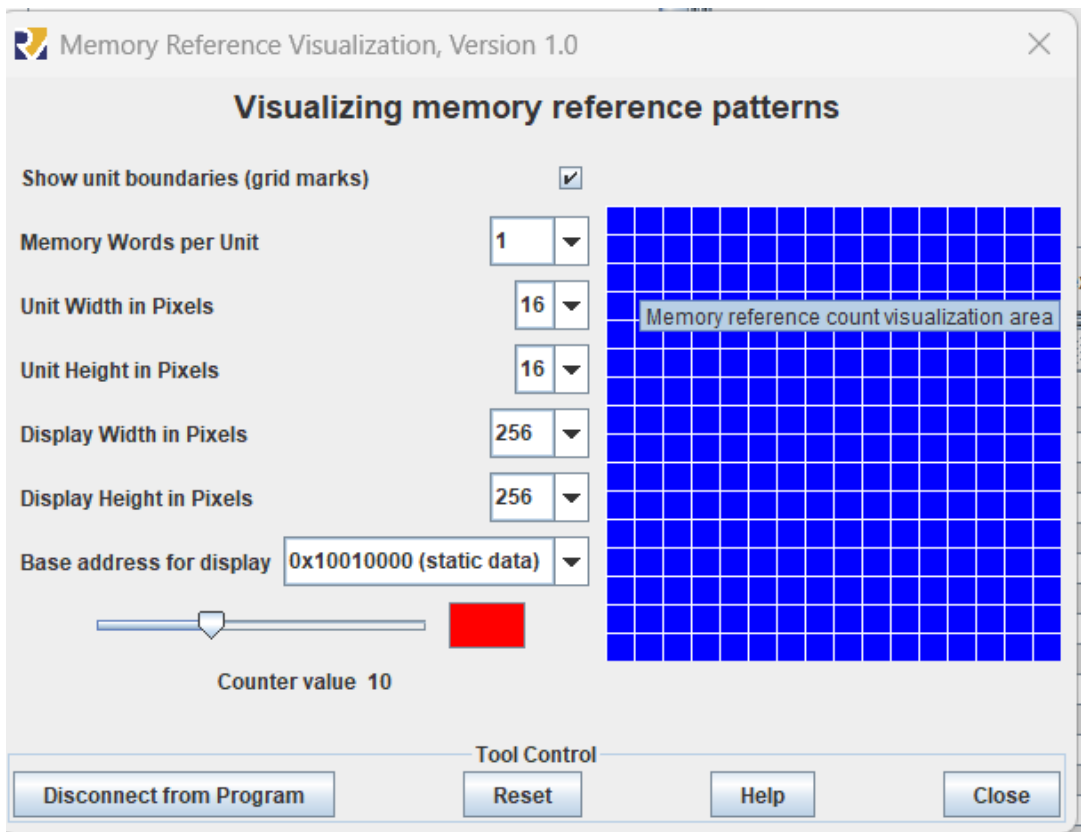
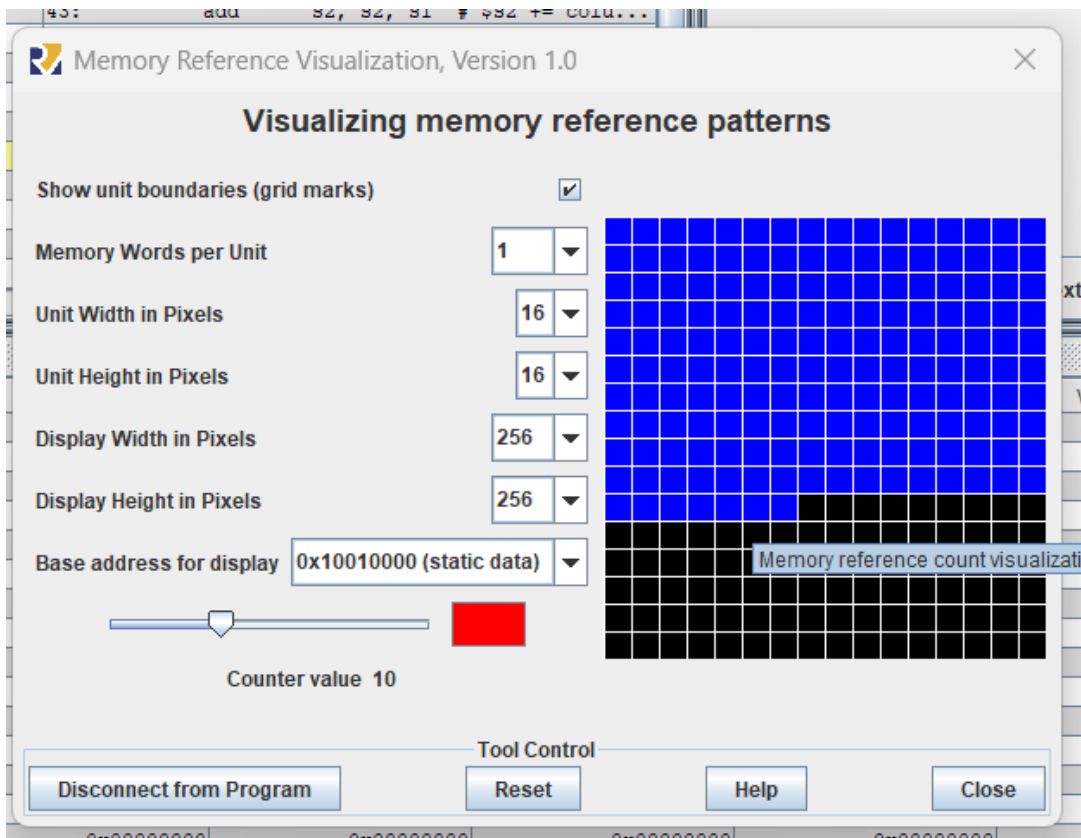
Kết quả và giải thích

1. Chương trình row-major.asm

Kết quả trực quan trên lưới:

- Các ô trên lưới được tô lần lượt từ trái sang phải theo từng hàng, sau đó mới chuyển xuống hàng kế tiếp. Hình ảnh tạo thành một vùng 16 x 16 được phủ màu xanh theo thứ tự liên mạch, phản ánh chuỗi địa chỉ bộ nhớ tăng đều từng word.





Phân tích hiện tượng:

Phân tích hiện tượng: Hình ảnh hiển thị trên lưới phản ánh đúng trình tự truy cập bộ nhớ của chương trình. Các ô được xử lý lần lượt từ trái sang phải rồi mới chuyển xuống hàng tiếp theo. Kiểu truy cập này tạo ra chuỗi địa chỉ liên tục, giúp dữ liệu vừa được nạp vào cache có xác suất cao được tái sử dụng ngay lập tức. Vì vậy đây là mẫu truy cập rất thân thiện với bộ nhớ đệm.

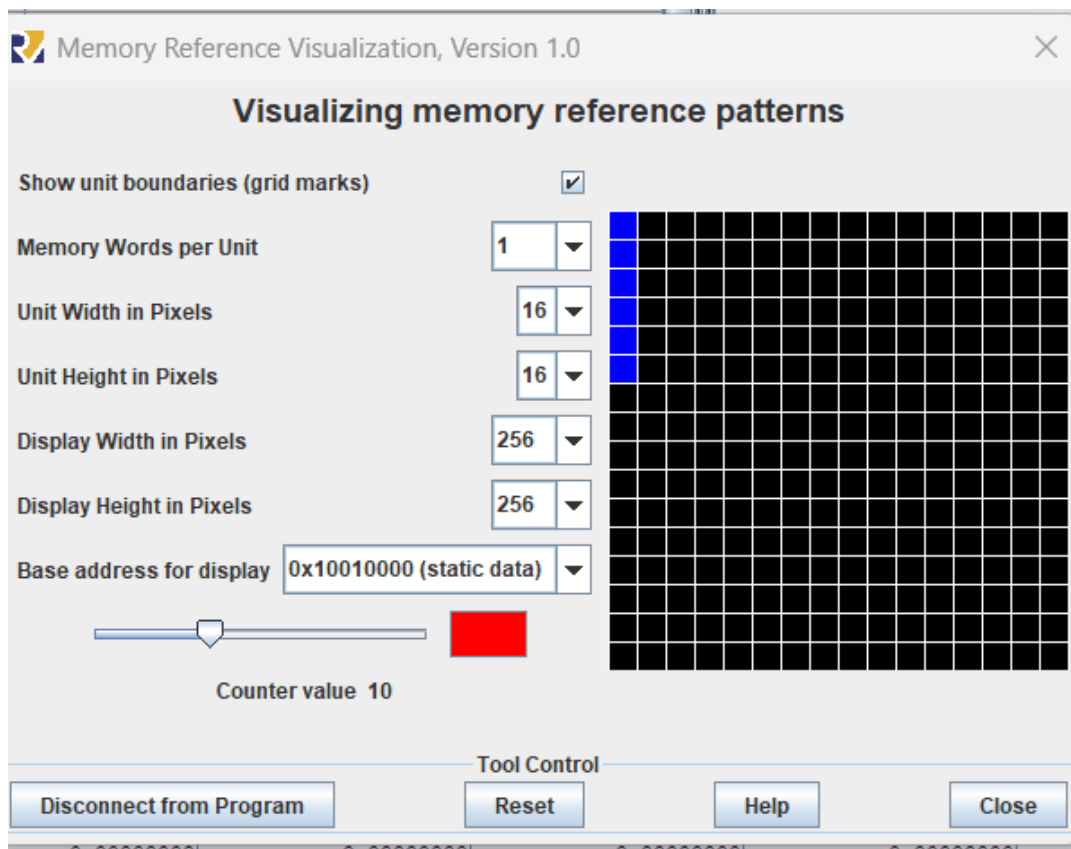
Do dữ liệu của mảng hai chiều được lưu tuyến tính theo row-major, thứ tự truy cập của chương trình khớp với thứ tự vật lý trong bộ nhớ. Mỗi lần ghi đi sang ô nhớ kế tiếp, nên bộ nhớ được tham chiếu theo dải địa chỉ liên tục. Đây là dạng truy cập có cục bộ không gian tốt, thuận lợi cho cache và cũng lý giải vì sao hình trực quan được tô thành từng hàng liền nhau.

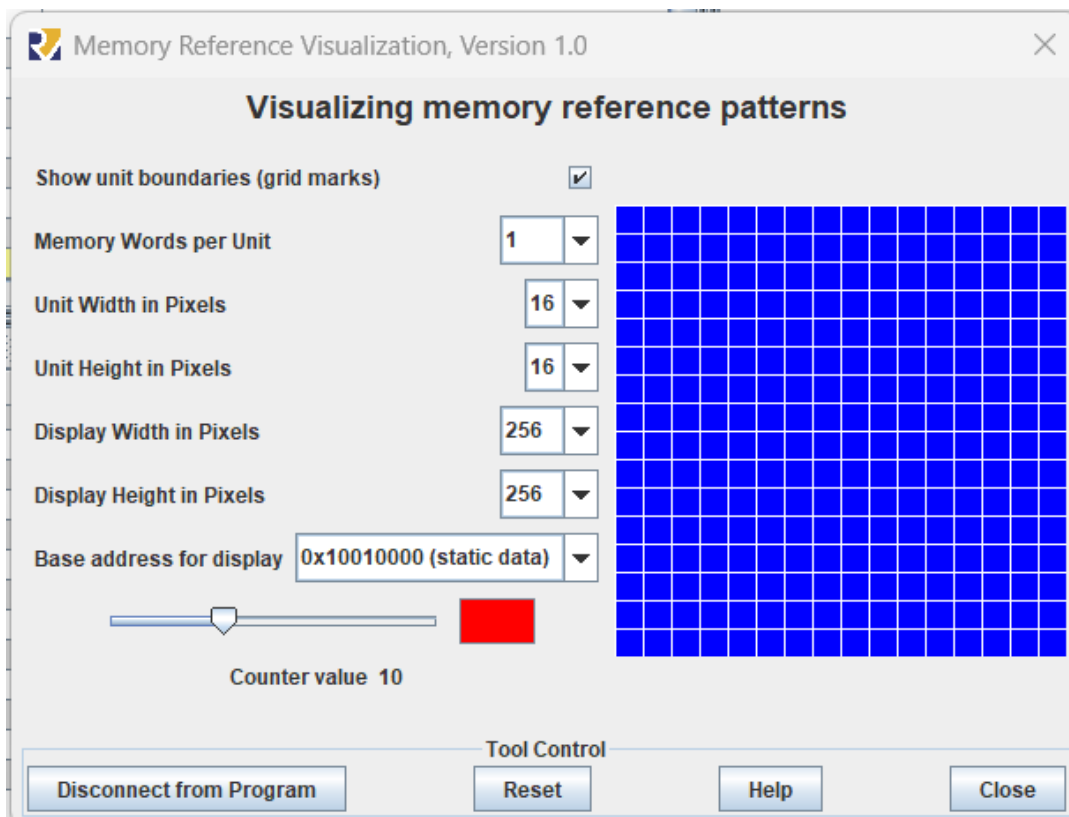
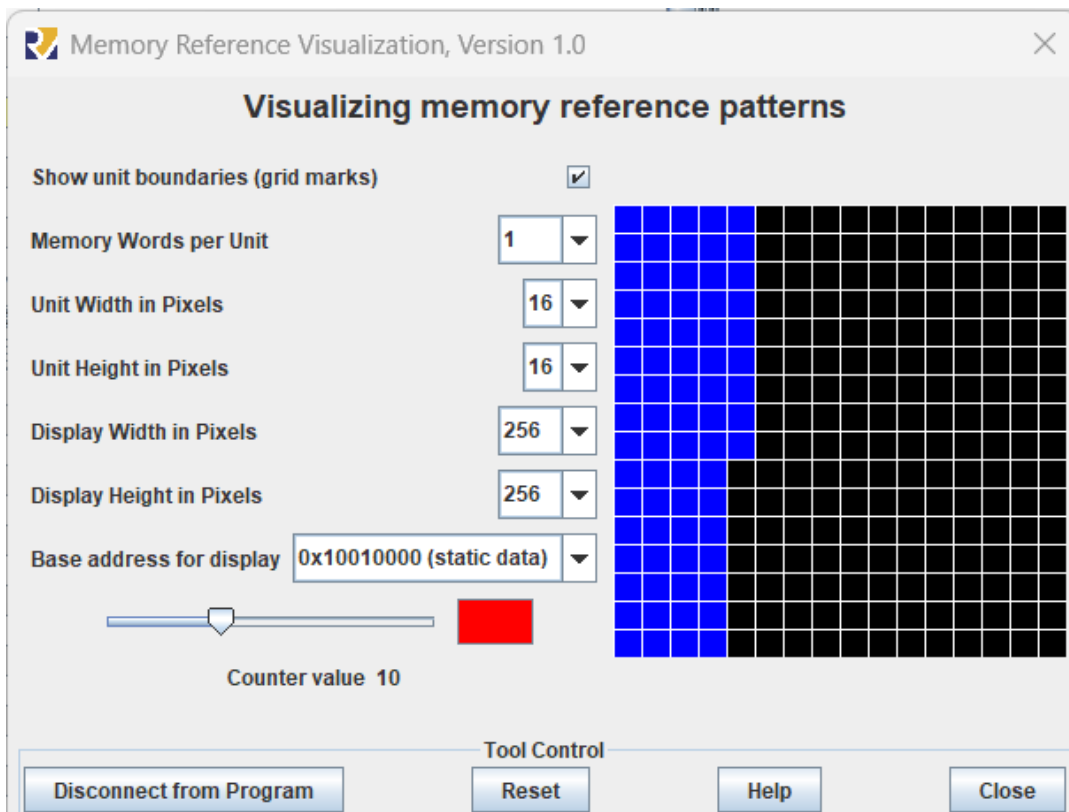
2. Chương trình column-major.asm

Kết quả trực quan trên lưới:

Thay vì xuất hiện liên tiếp theo chiều ngang, các ô được tô theo từng cột: sau khi tô một ô, chương trình nhảy xuống phần tử cùng cột ở hàng kế tiếp. Trên lưới, điều này tạo ra các vạch màu dọc trước khi chuyển sang cột bên cạnh.

- Khi hoàn tất một cột, chương trình quay lại vùng đầu ma trận để xử lý cột kế tiếp. Kết quả cuối cùng vẫn là khối 16 x 16, nhưng trình tự xuất hiện thể hiện rõ bước nhảy địa chỉ lớn giữa các lần truy cập liên tiếp.





Phân tích hiện tượng:

Phân tích hiện tượng: Mặc dù kết quả cuối cùng của chương trình không thay đổi, cách tổ chức vòng lặp đã làm thay đổi hoàn toàn hành vi truy cập bộ nhớ. CPU liên tục nhảy từ hàng này sang hàng khác thay

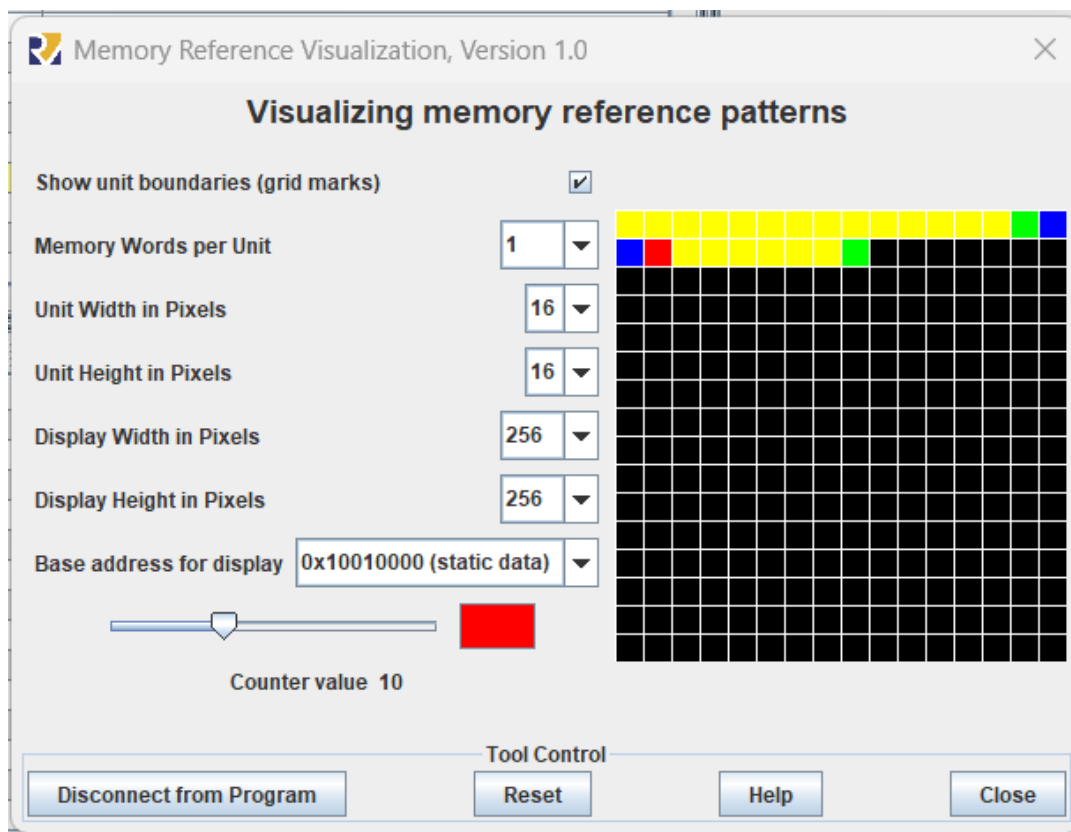
vì xử lý tuần tự từng hàng. Điều đó làm cho dữ liệu nằm trong cache ít được tái sử dụng, dẫn đến số lượng miss tăng lên và hiệu suất suy giảm đáng kể.

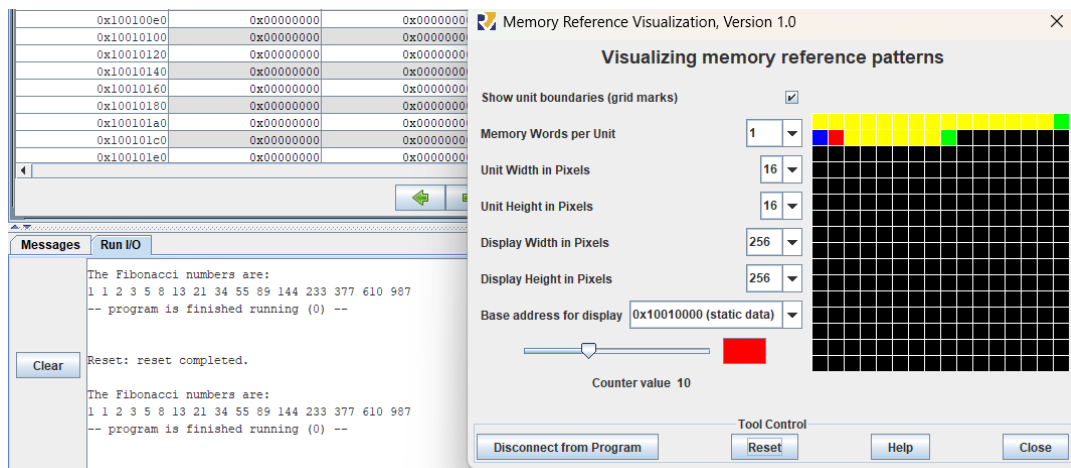
Vì mảng được lưu theo hàng, thay đổi row liên tục làm địa chỉ bộ nhớ nhảy qua cả một hàng, tương đương 16 words hoặc 64 bytes nếu mỗi word có kích thước 4 bytes. Kiểu truy cập stride lớn này làm giảm cục bộ không gian, khiến cache khó tận dụng dữ liệu vừa nạp và làm hiệu năng thấp hơn so với row-major.

3. Chương trình fibonacci.asm

Kết quả trực quan trên lưới:

- Chương trình fibonacci.asm không quét toàn bộ ma trận theo một mẫu đều đặn mà tập trung đọc và ghi trên một vùng nhớ nhỏ chứa các phần tử của dãy Fibonacci.
- Các ô ở đầu dãy được truy cập nhiều lần nên màu thay đổi liên tục theo mức độ tham chiếu:
- Xanh lam -> Xanh lá -> Vàng -> Cam -> Đỏ. Những ô phía sau được sử dụng ít hơn nên có màu nhạt hơn hoặc đổi màu chậm hơn.





Phân tích hiện tượng:

Phân tích hiện tượng: Thuật toán Fibonacci không tập trung vào việc quét toàn bộ vùng nhớ mà liên tục khai thác lại các giá trị vừa được tính toán. Những phần tử đầu dãy xuất hiện với tần suất rất cao trong các phép tính kế tiếp nên được truy cập lặp đi lặp lại. Điều này làm tăng đáng kể khả năng dữ liệu vẫn còn nằm trong cache tại thời điểm cần sử dụng, từ đó giảm số lần truy xuất bộ nhớ chính và nâng cao hiệu năng thực thi.

Vì các giá trị gần nhau trong dãy thường xuyên được dùng lại, chương trình có tính cục bộ thời gian rất rõ: cùng một ô nhớ được tham chiếu nhiều lần trong khoảng thời gian ngắn. Các ô đầu dãy vượt ngưỡng số lần truy cập của công cụ nên chuyển sang màu đỏ, chứng minh rằng mẫu truy cập của Fibonacci thiên về tái sử dụng dữ liệu hơn là quét tuần tự một lần như hai chương trình ma trận.